17 – REST API

Creating routes and behaviours for your website

Jérôme Landré – jerome.landre@ju.se

OUTLINE

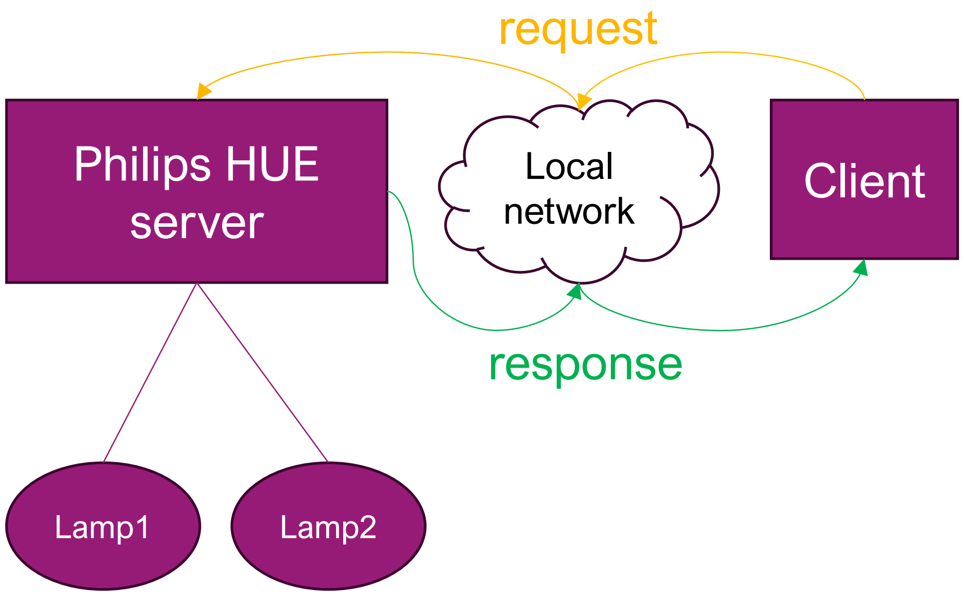
1) REST API
2) Example of a REST API: Philips HUE
3) List of REST APIs
4) Creating a REST API using Express

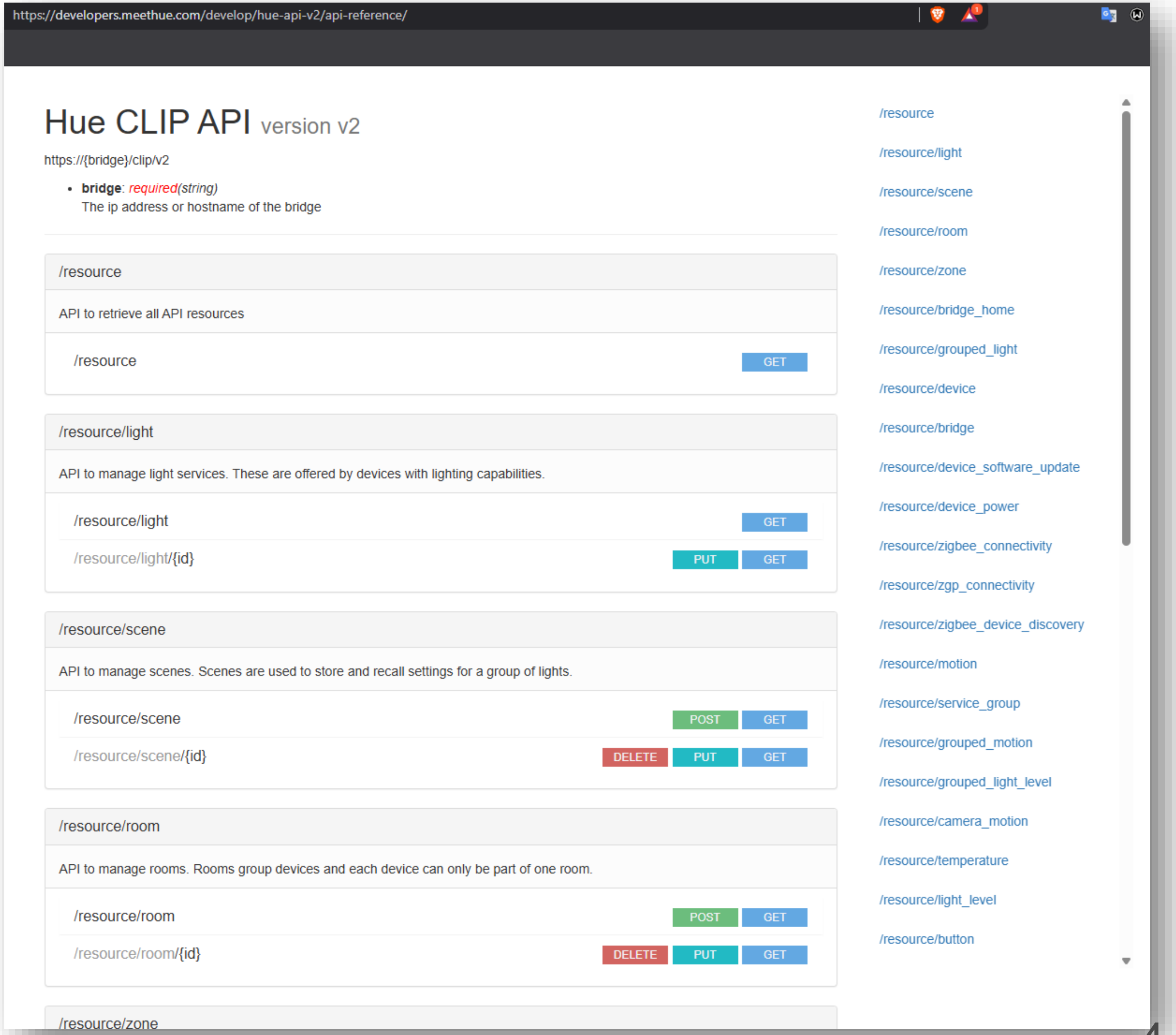
1) REST API

- A REST API (**REpresentational State Transfer** Application Programming Interface) is a way for different software systems to communicate over HTTP, using standard methods like GET, POST, PUT, and DELETE. REST APIs are stateless, meaning each request from a client contains all the information needed to process it.
- You define the routes for your API:
  - /api/v1.0/projects
  - /api/v1.0/projects/17
  - /api/v1.0/projects/new
  - /api/v1.0/projects/modify/17
  - /api/v1.0/projects/delete/17
  - …

Of course, you need to check:
Authentication (who is the user?)
and
Authorization (what functions are allowed for this user?)

2) EXAMPLE OF A REST API: PHILIPS HUE




https://developers.meethue.com/develop/hue-api-v2/api-reference/

Hue CLIP API version v2

https://{bridge}/clip/v2

- **bridge**: *required(string)*
  The ip address or hostname of the bridge

/resource

API to retrieve all API resources

/resource	GET

/resource/light

API to manage light services. These are offered by devices with lighting capabilities.

/resource/light	GET
/resource/light/{id}	PUT GET

/resource/scene

API to manage scenes. Scenes are used to store and recall settings for a group of lights.

/resource/scene	POST GET
/resource/scene/{id}	DELETE PUT GET

/resource/room

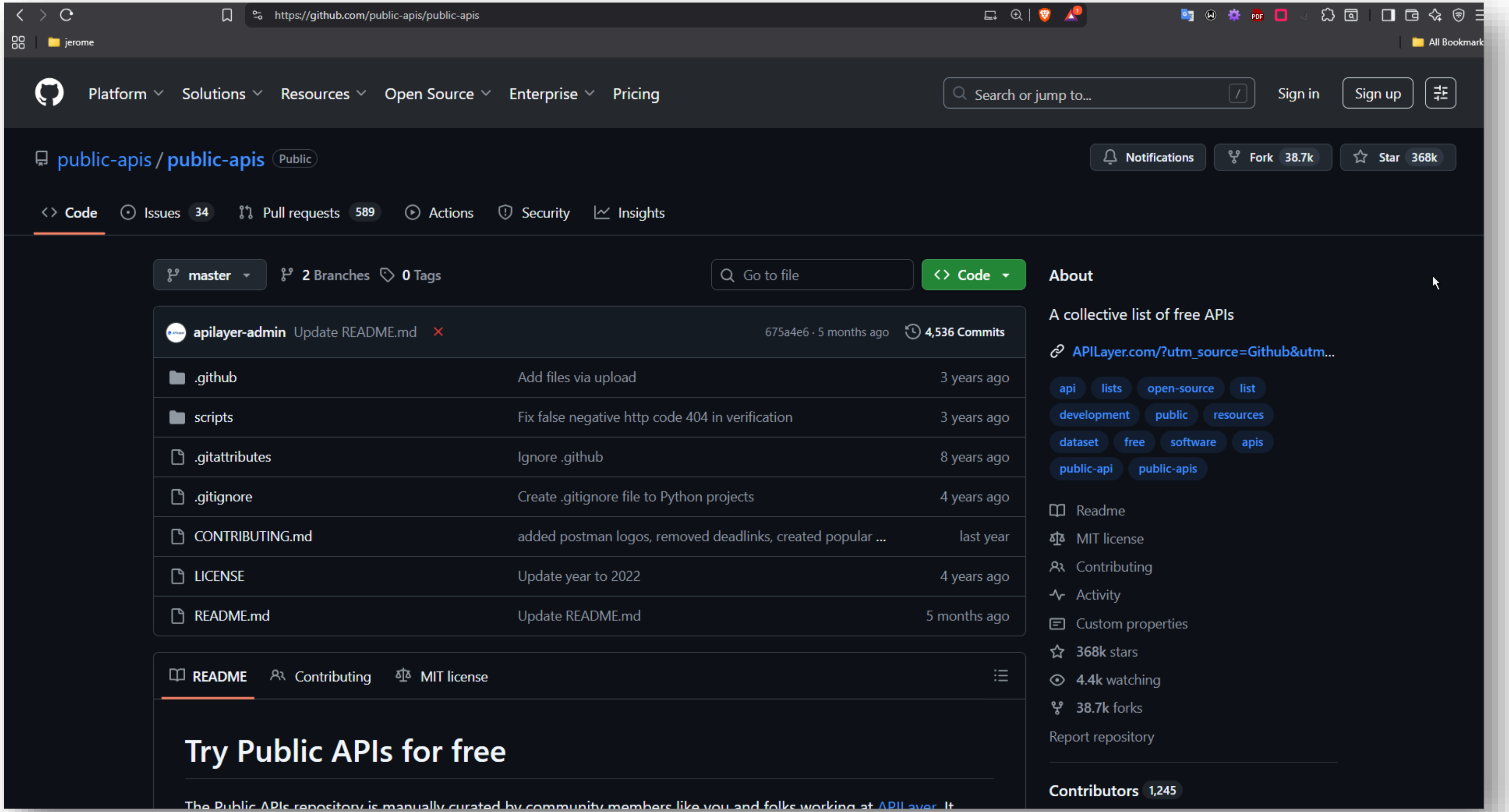
API to manage rooms. Rooms group devices and each device can only be part of one room.

/resource/room	POST GET
/resource/room/{id}	DELETE PUT GET

/resource/zone

- /resource
- /resource/light
- /resource/scene
- /resource/room
- /resource/zone
- /resource/bridge_home
- /resource/grouped_light
- /resource/device
- /resource/bridge
- /resource/device_software_update
- /resource/device_power
- /resource/zigbee_connectivity
- /resource/zgp_connectivity
- /resource/zigbee_device_discovery
- /resource/motion
- /resource/service_group
- /resource/grouped_motion
- /resource/grouped_light_level
- /resource/camera_motion
- /resource/temperature
- /resource/light_level
- /resource/button

3) LIST OF REST APIs



Source: https://github.com/public-apis/public-apis

3) LIST OF REST APIs

- Animals
- Anime
- Anti-Malware
- Art & Design
- Authentication & Authorization
- Blockchain
- Books
- Business
- Calendar
- Cloud Storage & File Sharing
- Continuous Integration
- Cryptocurrency
- Currency Exchange
- Data Validation
- Development
- Dictionaries
- Documents & Productivity
- Email
- Entertainment
- Environment
- Events
- Finance
- Food & Drink
- Games & Comics
- Geocoding

3) LIST OF REST APIs

Art & Design

API	Description	Auth	HTTPS	CORS
Améthyste	Generate images for Discord users	`apiKey`	Yes	Unknown
Art Institute of Chicago	Art	No	Yes	Yes
Colormind	Color scheme generator	No	No	Unknown
ColourLovers	Get various patterns, palettes and images	No	No	Unknown
Cooper Hewitt	Smithsonian Design Museum	`apiKey`	Yes	Unknown
Dribbble	Discover the world's top designers & creatives	`OAuth`	Yes	Unknown
EmojiHub	Get emojis by categories and groups	No	Yes	Yes
Europeana	European Museum and Galleries content	`apiKey`	Yes	Unknown
Harvard Art Museums	Art	`apiKey`	No	Unknown
Icon Horse	Favicons for any website, with fallbacks	No	Yes	Yes
Iconfinder	Icons	`apiKey`	Yes	Unknown
Icons8	Icons (find "search icon" hyperlink in page)	No	Yes	Unknown
Lordicon	Icons with predone Animations	No	Yes	Yes
Metropolitan Museum of Art	Met Museum of Art	No	Yes	No
Noun Project	Icons	`OAuth`	No	Unknown

3) LIST OF REST APIs

Art Institute of Chicago API

openapi.json GitHub

Documentation

- Introduction
  - Quick Start
  - Conventions
  - Best Practices
- Data Dumps
  - Data Dumps vs. API?
  - Scraping Data
- Images
  - IIIF Image API
  - Image Sizes
  - IIIF Manifests
  - Image Dumps
  - Scraping Images
  - Copyright
- General Information
  - Authentication
  - Pagination
- Endpoints
  - Collections
  - Shop
  - Mobile
  - Digital Scholarly Catalogs
  - Static Archive
  - Website
- Fields
  - Collections
  - Shop
  - Mobile
  - Digital Scholarly Catalogs
  - Static Archive

INTRODUCTION

The Art Institute of Chicago 's API provides JSON-formatted data as a REST-style service that allows developers to explore and integrate the museum's public data into their projects. This API is the same tool that powers our website , our mobile app , and many other technologies in the museum.

If the application you're building will be public, please send it our way! We'd love to share it alongside some of the other projects that use our API . And if you have any questions, please feel free to reach out to us: engineering@artic.edu.

Quick Start

An API is a structured way that one software application can talk to another. APIs power much of the software we use today, from the apps on our phones and watches to technology we see in sports and TV shows. We have built an API to let people like you easily get our data in an ongoing way.

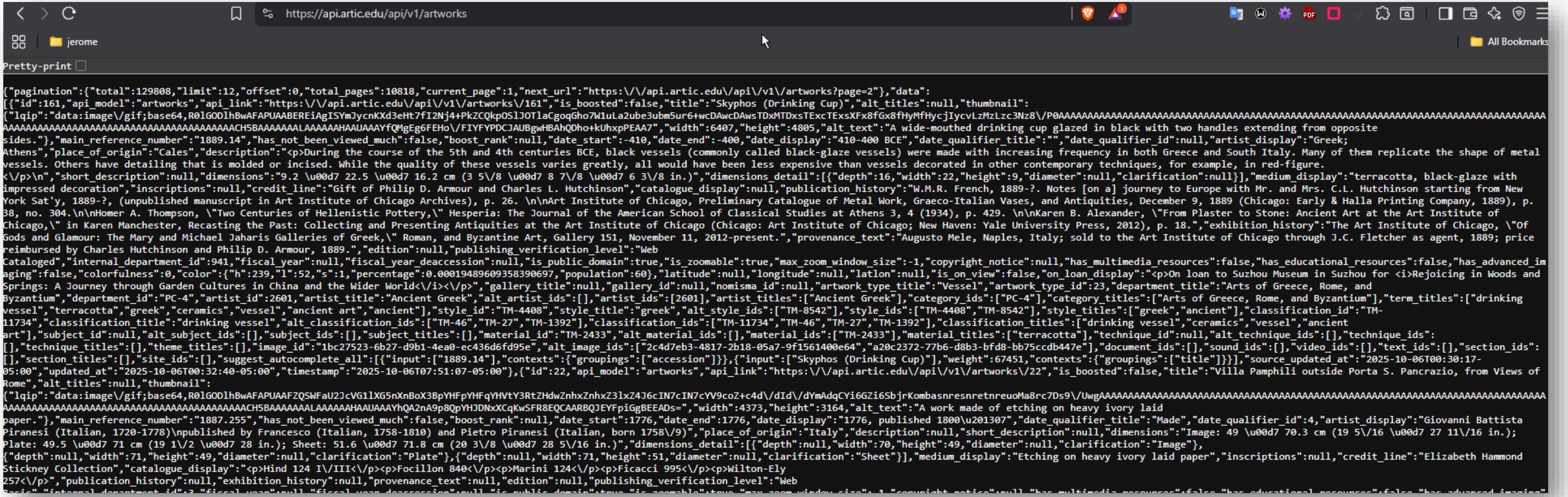
> **TIP**
>
> Do you want *all* of our data? Are you running into problems with throttling or deep pagination? Consider using our data dumps instead of our API.

For example, you can access the `/artworks` listing endpoint in our API by visiting the following URL to see all the published artworks in our collection:

```
https://api.artic.edu/api/v1/artworks
```

If you want to see data for just one artwork, you can use the `/artworks/{id}` detail endpoint.

3) LIST OF REST APIs



3) LIST OF REST APIs

Weather

API	Description	Auth	HTTPS	CORS
7Timer!	Weather, especially for Astroweather	No	No	Unknown
AccuWeather	Weather and forecast data	`apiKey`	No	Unknown
Aemet	Weather and forecast data from Spain	`apiKey`	Yes	Unknown
apilayer weatherstack	Real-Time & Historical World Weather Data API	`apiKey`	Yes	Unknown
APIXU	Weather	`apiKey`	Yes	Unknown
AQICN	Air Quality Index Data for over 1000 cities	`apiKey`	Yes	Unknown
AviationWeather	NOAA aviation weather forecasts and observations	No	Yes	Unknown
ColorfulClouds	Weather	`apiKey`	Yes	Yes
Euskalmet	Meteorological data of the Basque Country	`apiKey`	Yes	Unknown
Foreca	Weather	`OAuth`	Yes	Unknown
HG Weather	Provides weather forecast data for cities in Brazil	`apiKey`	Yes	Yes
Hong Kong Obervatory	Provide weather information, earthquake information, and climate data	No	Yes	Unknown
MetaWeather	Weather	No	Yes	No
Meteorologisk Institutt	Weather and climate data	`User-Agent`	Yes	Unknown
Micro Weather	Real time weather forecasts and historic data	`apiKey`	Yes	Unknown

4) REST API WITH EXPRESS


```javascript
// Sample data
let books = [
  { id: 1, title: 'Book 1', author: 'Author 1' },
  { id: 2, title: 'Book 2', author: 'Author 2' },
];

// REST API Endpoints
// GET all books
app.get('/api/books', (req, res) => {
  res.json(books);
});

// GET a single book by ID
app.get('/api/books/:id', (req, res) => {
  const book = books.find(b => b.id === parseInt(req.params.id));
  if (!book) return res.status(404).send('Book not found');
  res.json(book);
});
```

Code generated by Claude.ai: https://claude.ai/

4) REST API WITH EXPRESS


```javascript
// POST a new book
app.post('/api/books', (req, res) => {
  const newBook = { id: books.length + 1, title: req.body.title, author: req.body.author };
  books.push(newBook);
  res.status(201).json(newBook);
});

// PUT (update) a book
app.put('/api/books/:id', (req, res) => {
  const book = books.find(b => b.id === parseInt(req.params.id));
  if (!book) return res.status(404).send('Book not found');

  book.title = req.body.title;
  book.author = req.body.author;
  res.json(book);
});
```

Code generated by Claude.ai: https://claude.ai/

4) REST API WITH EXPRESS

```
// DELETE a book
app.delete('/api/books/:id', (req, res) => {
  const bookIndex = books.findIndex(b => b.id === parseInt(req.params.id));
  if (bookIndex === -1) return res.status(404).send('Book not found');

  books.splice(bookIndex, 1);
  res.status(204).send();
});

// Render a Handlebars view
app.get('/', (req, res) => {
  res.render('home', { books });
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Code generated by Claude.ai: https://claude.ai/

4) REST API WITH EXPRESS

```
<!DOCTYPE html>
<html>
<head>
  <title>Books API</title>
</head>
<body>
  {{{body}}}
</body>
</html>
```

```
<h1>Books</h1>
<ul>
  {{#each books}}
    <li>{{this.title}} by {{this.author}}</li>
  {{/each}}
</ul>
```

Code generated by Claude.ai: https://claude.ai/

4) REST API WITH EXPRESS

- You can test your REST API using:
  - **GET all books**: http://localhost:3000/api/books
  - **GET a single book**: http://localhost:3000/api/books/1
  - **POST a new book**: Send a POST request to http://localhost:3000/api/books with a JSON body like { "title": "Book 3", "author": "Author 3" }
  - **PUT (update) a book**: Send a PUT request to http://localhost:3000/api/books/1 with a JSON body like { "title": "Updated Book 1", "author": "Updated Author 1" }
  - **DELETE a book**: Send a DELETE request to http://localhost:3000/api/books/1
  - **View the Handlebars page**: http://localhost:3000/

Code generated by Claude.ai: https://claude.ai/

4) REST API WITH EXPRESS

- This lecture is **NOT** included in the **EXAMINATION** and you **DO NOT NEED** to write a REST API in your **project**

- It is just for **YOUR INFORMATION** to tell you what is a **REST API**!

ANY QUESTIONS?